Python para Ingeniería

Clase 01 - De ecuación matemática a algoritmo

Apuntes + ejemplos + ejercicios de trabajo

Objetivo de la clase: aprender a transformar un problema matemático en una secuencia lógica de entradas, operaciones, decisiones, iteraciones y salidas antes de escribir código.

1. La idea central

Resolver una ecuación en papel y programarla son habilidades relacionadas, pero no idénticas. Antes de escribir Python, hay que construir un **algoritmo**: una secuencia explícita de pasos que convierta entradas en salidas.

Problema → Algoritmo → Código → Validación

Casi cualquier programa científico combina cuatro estructuras:

1. **Secuencia**: hacer A, después B y después C.
2. **Decisión**: si ocurre una condición, tomar un camino; si no, tomar otro.
3. **Iteración**: repetir una operación un número de veces o hasta cumplir una condición.
4. **Descomposición**: dividir un problema grande en problemas pequeños que después se convertirán en funciones, módulos o clases.

Regla de trabajo: no empezar por Python. Empezar por definir qué entra, qué sale, qué ecuaciones intervienen y qué decisiones o repeticiones necesita el problema.

2. Ejemplo 1 - Presión dinámica

La presión dinámica viene dada por:

$$q = \frac{1}{2}\rho V^2$$

Entradas: ρ, V.

Salida: q.

Decisiones: ninguna para el cálculo básico.

Iteraciones: ninguna.

Algoritmo

```
START
Read rho
Read velocity
q = 0.5 * rho * velocity^2
Show q
END
```

Python

```python
rho = 1.225
velocity = 100.0

dynamic_pressure = 0.5 * rho * velocity**2

print(dynamic_pressure)
```

La primera mejora lógica aparece cuando preguntamos: *¿tiene sentido físico una densidad negativa?* Una validación física puede convertirse en un condicional.

3. Ejemplo 2 - Ecuación cuadrática

$$ax^2 + bx + c = 0$$

Definimos el discriminante:

$$\Delta = b^2 - 4ac$$

La matemática contiene tres casos distintos:

$$\Delta > 0, \qquad \Delta = 0, \qquad \Delta < 0$$

Por tanto el algoritmo necesita **ramificaciones**.

```
INPUT: a, b, c
Calculate discriminant

If discriminant > 0:
    calculate two real roots
Else if discriminant == 0:
    calculate one real root
Else:
    report no real roots

OUTPUT: roots or message
```

En Python estas ramas se expresan con:

```python
if condition:
    ...
elif another_condition:
    ...
else:
    ...
```

> **Los condicionales no son un artificio del lenguaje: representan bifurcaciones reales del razonamiento matemático o físico.**

4. Ejemplo 3 - Suma iterativa

Queremos calcular:

$$S = \sum_{i=1}^{N} i^2$$

Para $N = 4$:

$$S = 1^2 + 2^2 + 3^2 + 4^2$$

Ahora hay una operación que se repite. Necesitamos una variable acumuladora:

```
total = 0

for i in range(1, N + 1):
    total = total + i**2
```

Tras cada iteración se cumple conceptualmente:

$$\text{total}_{new} = \text{total}_{old} + i^2$$

Esto es especialmente importante en ingeniería porque muchas simulaciones tienen una estructura parecida:

$$x_{k+1} = x_k + v_k \Delta t$$
$$v_{k+1} = v_k + a_k \Delta t$$

5. Plantilla universal antes de programar

Usaremos siempre esta plantilla durante las primeras semanas:

```
PROBLEM
-------

INPUTS
------

OUTPUTS
-------

EQUATIONS
---------

SPECIAL CASES
-------------

DO I NEED ITERATION?
--------------------

ALGORITHM
---------

VALIDATION
----------
```

No es burocracia: obliga a separar la solución matemática de la sintaxis de Python.

6. Ejercicios de la Clase 1

Ejercicio 1 - Dynamic Pressure Calculator

Implementar:

$$q = \frac{1}{2}\rho V^2$$

Entradas: `rho`, `velocity`. Salida: `dynamic_pressure`.

Pensar también qué hacer si $\rho < 0$.

Ejercicio 2 - Rocket Equation

Implementar la ecuación de Tsiolkovsky:

$$\Delta V = I_{sp} g_0 \ln\left(\frac{m_0}{m_f}\right)$$

Entradas: `Isp`, `g0`, `initial_mass`, `final_mass`.

Identificar antes de programar qué relaciones físicas deben cumplir m_0 y m_f.

Ejercicio 3 - Quadratic Equation Solver

Resolver:

$$ax^2 + bx + c = 0$$

Considerar:

- $\Delta > 0$: dos raíces reales.
- $\Delta = 0$: una raíz real.
- $\Delta < 0$: mostrar `No real roots`.

No usar todavía números complejos.

Ejercicio 4 - Drag Coefficient

Modelo simplificado:

$$C_D = \begin{cases} 0{,}3 & M < 0{,}8 \\ 0{,}5 & 0{,}8 \leq M < 1{,}2 \\ 0{,}25 & M \geq 1{,}2 \end{cases}$$

Entrada: `Mach`. Salida: `Cd`. Dibujar mentalmente el árbol lógico antes de escribir el `if`.

Ejercicio 5 - Sum of Squares

Calcular iterativamente:

$$S = \sum_{i=1}^{N} i^2$$

Después validar contra:

$$S = \frac{N(N+1)(2N+1)}{6}$$

Ejercicio 6 - Factorial

Implementar:

$$N! = N(N-1)(N-2)\cdots 1$$

Sin usar `math.factorial()`.

Ejercicio 7 - Serie de Taylor de e^x

Aproximar:

$$e^x = \sum_{n=0}^{N} \frac{x^n}{n!}$$

Antes de programar identificar:

- qué se repite;
- qué cambia en cada iteración;
- qué valor se acumula;
- qué controla N.

Ejercicio 8 - Movimiento vertical

Sistema:

$$\dot{h} = v$$
$$\dot{v} = -g$$

Aproximación por pasos:

$$v_{k+1} = v_k - g\Delta t$$
$$h_{k+1} = h_k + v_k\Delta t$$

Repetir mientras la altitud sea no negativa. Este será el primer puente hacia simulación dinámica.

7. Mini Project 01 - Ballistic Flight Simulator

Primera versión:

- movimiento vertical;
- gravedad constante;
- sin resistencia aerodinámica;
- paso temporal fijo.

Evolución prevista:

Versión	Mejora
V1	Gravedad constante
V2	Gravedad dependiente de la altitud
V3	Densidad atmosférica
V4	Resistencia aerodinámica
V5	Masa variable
V6	Empuje
V7	Trayectoria de cohete

La idea es aprender desarrollo incremental: construir primero un modelo pequeño y verificable, y añadir complejidad de forma controlada.

8. Organización de la carpeta de esta clase

```
Python_Engineering/
‘---- 01_python_bootcamp/
    ‘---- week_01_logic/
        ‘---- class_01_equation_to_algorithm/
            |---- notes/
            |---- exercises/
            |   |---- ex01_dynamic_pressure.py
            |   |---- ex02_rocket_equation.py
            |   |---- ex03_quadratic_solver.py
            |   |---- ex04_drag_coefficient.py
            |   |---- ex05_sum_squares.py
            |   |---- ex06_factorial.py
            |   |---- ex07_taylor_exp.py
            |   ‘---- ex08_vertical_motion.py
            ‘---- mini_project/
                ‘---- ballistic_flight_simulator.py
```

9. Primera entrega para revisión

Empezar únicamente con los ejercicios 1, 2, 3 y 4. Para cada uno escribir primero:

```
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
ALGORITHM
```

Después escribir el código Python.

La revisión seguirá este orden:

Matemática → Algoritmo → Lógica → Python → Estilo → Casos límite

No buscar la solución antes de intentarlo. Un error propio es información útil: permite detectar exactamente qué parte del razonamiento necesitamos reforzar.

Ecuación → Algoritmo → Código

Ese es el hábito que vamos a construir durante todo el programa.